

Chapitre 6 : Les chaînes de caractères

Le type caractère

Le **type caractère** est utilisé pour représenter un seul symbole dans un programme. Il s'applique à tous les caractères définis dans le **code ASCII** (American Standard Code for Information Interchange).

Catégories de caractères ASCII :

- **Les lettres :**
 - Majuscules : 'A' à 'Z'
 - Minuscules : 'a' à 'z'
- **Les chiffres :**
 - '0' à '9'
- **Les caractères spéciaux :**
 - Exemples : '/', '*', '?', '&', etc.
- **Les caractères de contrôle :**
 - Exemples : <Retour Chariot>, <Échap>, etc.
- **Caractéristique principale :**
 - Chaque caractère est associé à un **numéro unique** (code ASCII) compris entre **0 et 255**.
 - Ce code permet au programme de manipuler les caractères facilement, par exemple pour les comparer, les trier ou les convertir

Fonctions standards sur les caractères

Fonction	Rôle	Exemple
ord(c)	retourne le code ASCII du caractère c	i ← ord ('A') ☞ i contiendra 65
Chr(i)	retourne le caractère dont le code ASCII est égal à i.	c ← Chr (65) ☞ c contiendra 'A'
Succ(c)	retourne le successeur du caractère c, s'il existe	c ← Succ('a') ☞ c contiendra 'b'
Pred(c)	retourne le prédécesseur du caractère c, s'il existe	c ← Pred('b') ☞ c contiendra 'a'
Majus(c)	retourne le majuscule du caractère c	c ← Majus('a') ☞ c contiendra 'A'
Minus(c)	retourne le minuscule du caractère c	c ← Minus ('A') ☞ c contiendra 'a'
EstMajuscule(c)	retourne <i>vrai</i> si le caractère c est une majuscule et <i>faux</i> sinon	b ← EstMajuscule ('A') ☞ b contiendra vrai
EstMinuscule(c)	retourne <i>vrai</i> si le caractère c est une minuscule et <i>faux</i> sinon	b ← EstMinuscule ('A') ☞ b contiendra faux

Fonctions standards sur les caractères :

Exemple 1

écrire un algorithme qui lit un caractère au clavier puis affiche son prédécesseur, son successeur et son équivalent en majuscule.

Algorithme Caract

Variables

c : Caractère

Début

Ecrire("Entrer un caractère: ")

Lire(c)

Ecrire(pred(c))

Ecrire(succ(c))

Ecrire(ord(Majusc(c)))

Fin.

Fonctions standards sur les caractères :

Exemple 2

Algorithme Majus

Variables

c1, c2 : Caractère

Début

Ecrire("Entrer un caractère: ")

Lire(c1)

Si (c1 >= 'a' et c1 <= 'z') **Alors**

 c2 ← chr(ord(c1)- 32)

Sinon

 c2 ← c1

FinSi

Ecrire(c2)

Fin.

Algorithme Minus

Variables

c1, c2 : Caractère

Début

Ecrire("Entrer un caractère: ")

Lire(c1)

Si (c1 >= 'A' et c1 <= 'Z') **Alors**

 c2 ← chr(ord(c1)+ 32)

Sinon

 c2 ← c1

FinSi

Ecrire(c2)

Fin.

Fonctions standards sur les caractères :

Exemple 3

Ecrire un algorithme qui affiche une table ASCII des lettres minuscules sous la forme suivante:

Le code ASCII de a est 97

Le code ASCII de b est 98

...

Le code ASCII de z est 122

Algorithme table_minus

Variables

c : Caractère

Début

Pour c de 'a' à 'z' **Faire**

Ecrire("Le code ASCII de ",c, " est ", ord(c))

FinPour

Fin.

Fonctions standards sur les caractères :

Exemple 4

Ecrire un algorithme qui lit une lettre au clavier puis affiche s'il s'agit d'une consonne ou d'une voyelle.

Algorithme Cons_Voy

Variables

c : Caractère

Début

Répéter

Ecrire("Entrer un caractère : ") Lire(c)

Ou bien

Jusqu'à (EstMajuscule(c) ou EstMinuscule(c))

Jusqu'à (Majus(c) >= 'A' et Majus(c) <= 'Z')

Si (Majus(c)='A') ou (Majus(c)='E') ou (Majus(c)='I')

ou (Majus(c)='O') ou (Majus(c)='U') ou (Majus(c)='Y') **Alors**

Ecrire(c, " est une voyelle")

Sinon

Ecrire(c, " est une consonne")

FinSi

Fin.

Les chaînes de caractères

Déclaration d'une chaîne :

Variables :

ch : Chaîne[20]



peut contenir au maximum 20 caractères

Opérations sur les chaînes de caractères :

La concaténation:

La concaténation est l'assemblage de deux chaînes de caractères en utilisant l'opérateur « + » ou « & »

Exemple :

```
Chn1 <- " turbo "
```

```
Chn2 <- "pascal"
```

```
Chn3 <- chn1+ " " +chn2
```

=> La variable chn3 contiendra "turbo pascal"

Opérations sur les chaînes de caractères :

Les opérateurs relationnels (>, >=, <=, =, ≠)

Il est possible d'effectuer une comparaison entre deux chaînes de caractères, le résultat est de type booléen. La comparaison se fait caractère par caractère de la gauche vers la droite selon le code ASCII

Exemples :

- L'expression ('a' > 'A') est vraie puisque le code ASCII de 'a' (97) est supérieur à celui de 'A' (65)
- L'expression ("programme" < "programmation") est fausse puisque 'e' > 'a'
- L'expression (" = ' ') est fausse (le vide est différent du caractère espace)
- L'expression ("chat" < "chaton") est vraie puisque la chaîne "chat" est le début de "chaton"

Opérations sur les chaînes de caractères :

Taille d'une chaîne

La taille d'une chaîne peut évoluer. Pour accéder à la taille d'une chaîne, nous utilisons la fonction `long()`.

Exemples :

```
ch : Chaîne      L: entier  
Ch <- "Bonjour "  L <- long(ch) //L=7
```

- La chaîne ne contenant aucun caractère est appelée chaîne vide.
- Pour tester si une chaîne est vide ou non on peut utiliser la fonction **`vide()`** qui **retourne vrai si la chaîne est vide, faux si non**
Exemple : `vide(ch)` : retourne faux
- Pour effacer le contenu d'une chaîne on peut utiliser la fonction **`efface()`**. Exemple : `efface(ch)` : **retourne la chaîne vide ch**

Opérations sur les chaînes de caractères :

Accès à un caractère dans une chaîne

Pour accéder à un caractère d'une chaîne, il suffit d'indiquer le nom de la chaîne suivi d'un entier entre crochets qui indique la position du caractère dans la chaîne, comme pour les tableaux.

Exemples :

```
chn <- "Turbo Pascal "
```

```
c    <- chn[7]
```

=> la variable c, de type caractère, contiendra le caractère 'P'

Les chaînes de caractères

Exercice

Ecrire un algorithme «Palind» qui lit une chaîne de caractères et vérifie si cette chaîne est un palindrome ou non. Un palindrome est un mot qui peut être lu indifféremment de droite à gauche ou de gauche à droite

(Exemples: "AZIZA", "LAVAL", "RADAR", "2002", etc.)

Algorithme Palind

Variables

ch : Chaîne
i, L : Entier
Pal : Booléen

Début

Ecrire("Entrer une chaîne non vide : ")
Lire(ch)
 $L \leftarrow \text{long}(\text{ch})$
 $\text{Pal} \leftarrow \text{Vrai}$
 $i \leftarrow 1$

TantQue ($i \leq L \text{ div } 2$) et (Pal) **Faire**

Si ($\text{ch}[i] = \text{ch}[L-i+1]$) **Alors**

$i \leftarrow i + 1$

Sinon

$\text{Pal} \leftarrow \text{Faux}$

FinSi

FinTQ

Si (Pal) **Alors**

 Ecrire(ch, " est un palindrome")

Sinon

 Ecrire(ch, " n'est pas un palindrome")

FinSi

Fin.

Les chaînes de caractères

Nom	Rôle	Exemple
Convch (N, Ch)	Convertit une valeur numérique N en une chaîne Ch.	Convch (2021, CH) ☞ CH = "2021"
Valeur (Ch, N, err)	Convertit une chaîne de caractères Ch en une valeur numérique N. De plus, elle fournit un code d'erreur ERR qui indique si l'opération s'est déroulée correctement.	Valeur ("19", n, err) ☞ n = 19 et err = 0 Valeur ("25H64", n, err) ☞ n = 0 et err = 3
EffaceN(Ch, P, N)	Enlève N caractères de Ch à partir de la position P donnée.	chn ← "Turbo Pascal" EffaceN(chn,6,7) ☞ chn contiendra "Turbo"
Insert(Ch1, Ch2, P)	Insère la chaîne Ch1 dans la chaîne Ch2 à partir de la position P.	ch1 ← "D" ch2 ← "AA" insert(ch1,ch2,2) ☞ ch2 contiendra "ADA"

Les chaînes de caractères

Nom	Rôle	Exemple
Sous_chaîne (ch, p, n)	Retourne une sous chaîne de n caractères à partir de la position p de la chaîne ch.	CH $\leftarrow$ Sous_chaîne("Langage", 4, 3) ☞ CH = "gag"
Pos (ch1, ch2)	Retourne la première position de la chaîne ch1 dans la chaîne ch2.	P $\leftarrow$ Pos ("a", "Programmation") ☞ P = 6 P $\leftarrow$ Pos ("gra", "Program") ☞ P = 4 P $\leftarrow$ Pos ("R", "Program") ☞ P = 0
Concat (ch1, ch2, ...)	Retourne la concaténation de plusieurs chaînes en une seule. C'est l'équivalent de ch1+ch2+...	CH1 $\leftarrow$ "Décembre" CH2 $\leftarrow$ Concat ("14", "/", ch1, "/", "2021") ☞ CH2 = "14/Décembre/2021"

Exercice

Écrire une fonction qui, pour une chaîne S donnée, **compte le nombre de voyelles** qu'elle contient

- Entrée : une chaîne de caractères S.
- Sortie : le nombre de voyelles dans la chaîne.

Fonction nbr_voyelles(ch:chaîne)

Début

cp <- 0

pour i **de** 1 **à** long(ch) **Faire**

si ch[i] == 'A' ou ch[i] == 'E' ou ch[i] == 'I' ou ch[i] == 'O' ou ch[i] == 'U' ou ch[i] == 'a' ou ch[i] == 'e' ou ch[i] == 'i' ou ch[i] == 'o' ou ch[i] == 'u'

alors

cp <- cp + 1

Finsi

retourner cp

Fin